tables and code

Paperflow Test Suite

November 2026

1 Computational Benchmarks & Algorithm Specification

Author: High-Performance Computing Group Preset: Technical Documentation

1.1 1. Algorithmic Complexity

The asymptotic computational complexity of matrix operations:

Algorithm	Best Case	Average Case	Worst Case	Memory
Gaussian Elimination	$\mathcal{O}(n^3)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^2)$
Strassen Matrix Multiplication	$\mathcal{O}(n^{2.807})$	$\mathcal{O}(n^{2.807})$	$\mathcal{O}(n^{2.807})$	$\mathcal{O}(n^2)$
Coppersmith–Winograd	$\mathcal{O}(n^{2.373})$	$\mathcal{O}(n^{2.373})$	$\mathcal{O}(n^{2.373})$	$\mathcal{O}(n^2)$
Fast Fourier Transform (FFT)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$

1.2 2. Implementation in Python

```
import numpy as np

def fast_fourier_transform(x: np.ndarray) -> np.ndarray:
    """
    Cooley-Tukey Radix-2 Decimation-in-Time FFT algorithm.
    Recursively computes the discrete Fourier transform of a 1D vector.
    """
    N = len(x)
    if N <= 1:
        return x
```

```
    even = fast_fourier_transform(x[0::2])
    odd = fast_fourier_transform(x[1::2])
    factor = np.exp(-2j * np.pi * np.arange(N) / N)
    return np.concatenate([
        even + factor[:N // 2] * odd,
        even + factor[N // 2:] * odd
    ])
```

1.3 3. Rust Kernel Optimization

```
pub fn vector_dot_product(a: &[f64], b: &[f64]) -> f64 {
    assert_eq!(a.len(), b.len(), "Vectors must have identical dimensions");
    a.iter().zip(b.iter()).map(|(&x, &y)| x * y).sum()
}
```